

1. Introduction to Control Structures

What are Control Structures?

Control structures are fundamental programming constructs that **determine the flow of execution** in a program. They decide **which parts of the code are executed**, how many times, and under what conditions.

Why are Control Structures Important?

- They enable programs to **make decisions** (choose between different paths).
 - They allow **repetition** of certain actions (looping).
 - They provide mechanisms to **jump** or skip code segments when necessary.
 - Without control structures, a program would simply execute instructions one after another without flexibility.
-

Main Types of Control Structures

1. **Decision Making (Conditional Statements):**
 - Allows the program to choose between different actions based on conditions.
 - Examples: `if`, `if-else`, `switch-case`.
2. **Looping (Iteration):**
 - Repeats a block of code multiple times until a condition is met.
 - Examples: `for`, `while`, `do-while`.
3. **Jump Statements:**
 - Alters the normal sequential flow by jumping to different parts of code.
 - Examples: `break`, `continue`, `goto` (less commonly used).

2. Types of Control Structures

Control structures control the flow of a program by deciding which blocks of code to execute, how many times to repeat them, or when to jump to other parts. They are broadly classified into **Branching, Looping, Jumping, and Nested Control Structures**.

A. Branching Statements

Branching statements allow the program to make decisions and execute specific code blocks based on conditions.

a. if Statement

- Executes a block of code **only if** the condition is true.

```
c
if (condition) {
    // code to execute if condition is true
}
```

b. if-else Statement

- Chooses between **two** blocks depending on the condition.

```
c
if (condition) {
    // true block
} else {
    // false block
}
```

c. if-else if-else Ladder

- Handles **multiple conditions**, checking them one by one until one is true.

```
c
if (condition1) {
    // block 1
} else if (condition2) {
    // block 2
} else {
    // default block if all conditions fail
}
```

d. switch-case Statement

- Used to compare a variable against **multiple possible values** and execute the corresponding block.

```
c
switch (expression) {
```

```
case value1:
    // code for value1
    break;
case value2:
    // code for value2
    break;
default:
    // default code if no case matches
}
```

- The `break` statement prevents "fall-through" to subsequent cases.
-

B. Looping Statements

Looping statements allow execution of a block repeatedly while a condition holds.

a. while Loop

- Tests the condition **before** executing the loop body.
- If the condition is false initially, the loop body may never execute.

c

```
while (condition) {
    // repeated code
}
```

b. do-while Loop

- Executes the loop body **at least once** before checking the condition.
- Useful when the loop must run at least one time.

c

```
do {
    // repeated code
} while (condition);
```

c. for Loop

- Best used when the **number of repetitions is known** beforehand.
- Includes initialization, condition check, and increment/decrement in one line.

c

```
for (initialization; condition; increment) {  
    // repeated code  
}
```

C. Jumping Statements

Jumping statements alter the normal sequential flow of the program.

a. goto Statement

- Jumps to a labeled statement in the code.
- Considered harmful and discouraged because it makes code harder to read and maintain.

```
c  
  
goto label;  
// other code  
label:  
    // code here
```

b. break Statement

- Exits the **current loop or switch** immediately.

```
c  
  
for (...) {  
    if (condition) break; // exit loop if condition is true  
}
```

c. continue Statement

- Skips the **rest of the current loop iteration** and moves to the next iteration.

```
c  
  
for (...) {  
    if (condition) continue; // skip current iteration if condition is true  
}
```

D. Nested Control Structures

Control structures can be placed **inside** other control structures to create complex logic.

Example:

```
c
for (int i = 0; i < 3; i++) {
    if (i % 2 == 0) {
        printf("Even\n");
    }
}
```

- A `for` loop runs from 0 to 2.
- Inside the loop, an `if` statement checks if `i` is even and prints "Even".

3. Example Program Using All Structures

```
c
#include <stdio.h>

int main() {
    int i; // Declare an integer variable i

    for (i = 1; i <= 5; i++) { // Loop from 1 to 5
        if (i == 3) { // Check if i equals 3
            continue; // Skip the rest of this iteration when i is
3
        }
        printf("%d\n", i); // Print the current value of i
    }

    return 0; // End of program
}
```